Objective

- Under-the-hood view of ML training methods

- Target ML models
  - Linear and Polynomial Regression

- Closed-form equations
- Gradient Descend (GD)
  - Batch GD
  - Mini-batch GD
  - Stochastic GD

Linear Regression

Equation 4-1. Linear Regression model prediction

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Equation 4-2. Linear Regression model prediction (vectorized form)

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x}$$

The dot product of two vectors $\mathbf{a} = [a_1, a_2, \ldots, a_n]$ and $\mathbf{b} = [b_1, b_2, \ldots, b_n]$ is defined as:[2]

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i-1}^{n} a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

Wikipedia

Linear Regression - Training

Equation 2-1. Root Mean Square Error (RMSE)

$$\mathrm{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)^2}$$

Equation 4-3. MSE cost function for a Linear Regression model

$$\mathrm{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m}\sum_{i=1}^{m}\left(\boldsymbol{\theta}^T\mathbf{x}^{(i)} - y^{(i)}\right)^2$$

The Normal Equation

Equation 4-4. Normal Equation

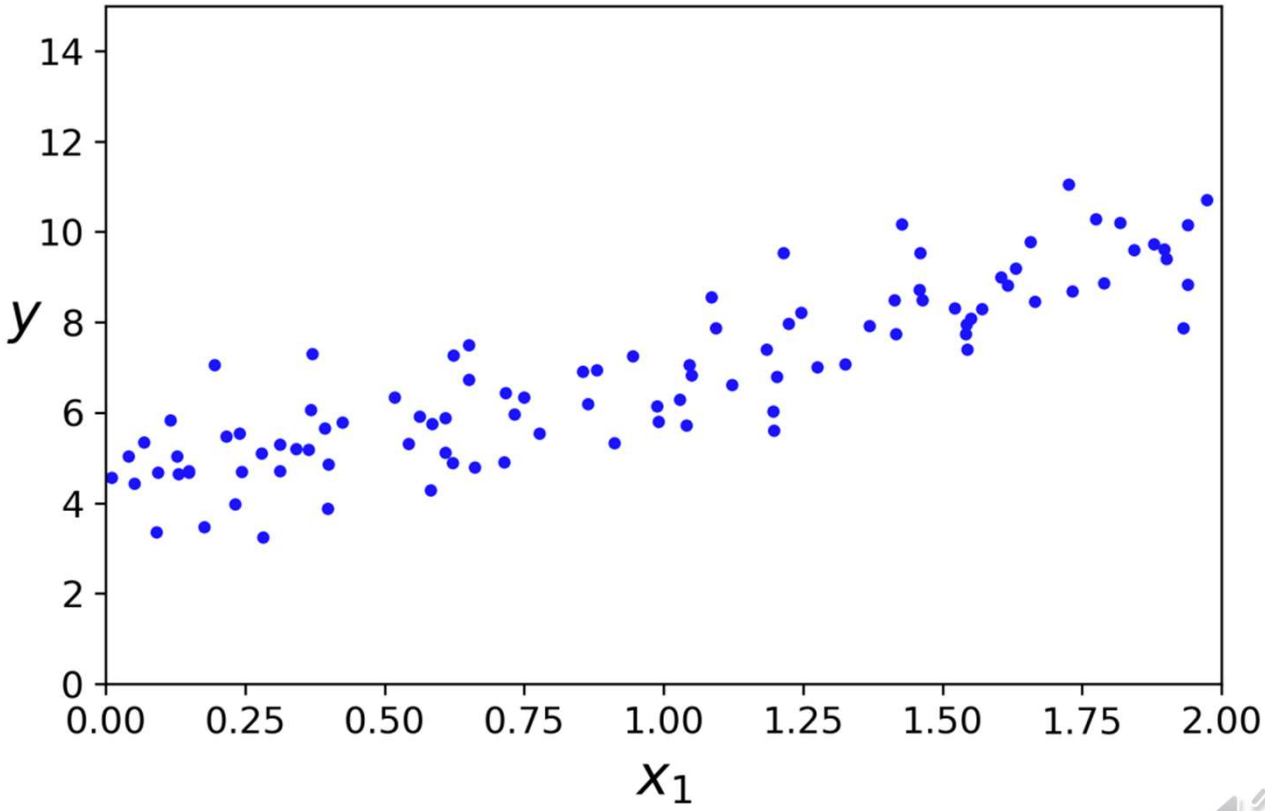
$$\widehat{\boldsymbol{\theta}} = \left(\mathbf{X}^T\mathbf{X}\right)^{-1} \quad \mathbf{X}^T \quad \mathbf{y}$$

- $\widehat{\boldsymbol{\theta}}$ is the value of $\boldsymbol{\theta}$ that minimizes the cost function.
- $\mathbf{y}$ is the vector of target values containing $y^{(1)}$ to $y^{(m)}$.

The Normal Equation – Python Example

```
import numpy as np

X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)
```



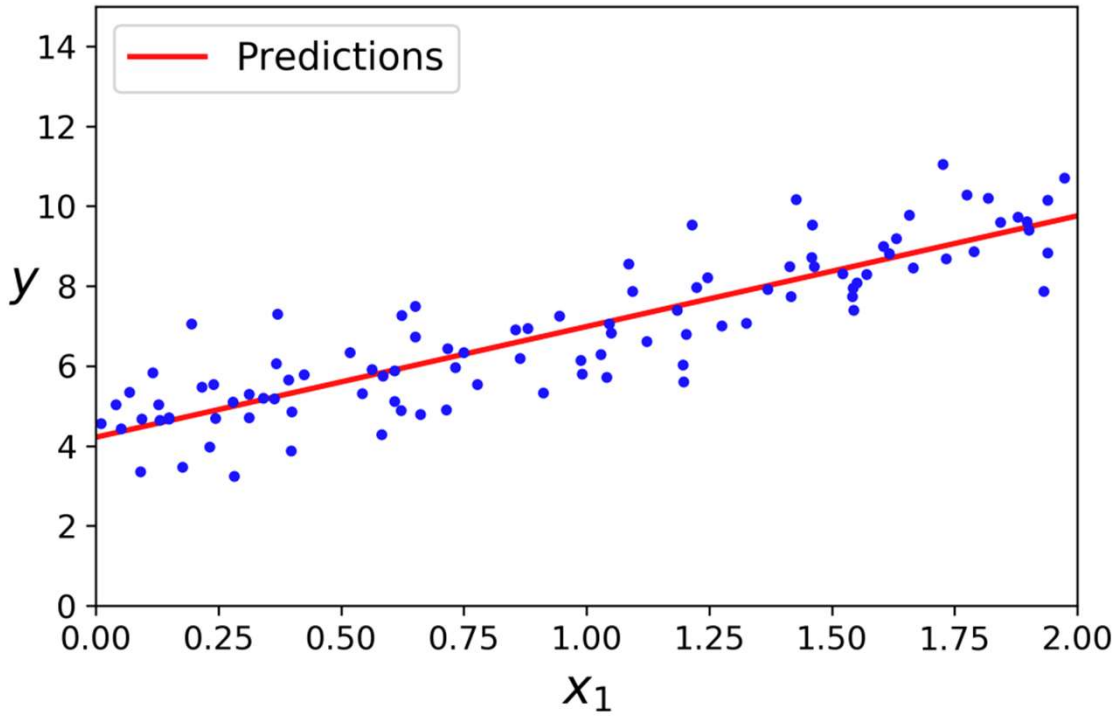
The Normal Equation – Python Example

```
X_b = np.c_[np.ones((100, 1)), X]  # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
```

```
>>> theta_best
array([[4.21509616],
       [2.77011339]])
```

The Normal Equation – Python Example

```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = np.c_[np.ones((2, 1)), X_new] # add x0 = 1 to each instance
>>> y_predict = X_new_b.dot(theta_best)
>>> y_predict
array([[4.21509616],
       [9.75532293]])
```



Linear Regression with Scikit-Learn

```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([4.21509616]), array([[2.77011339]]))
>>> lin_reg.predict(X_new)
array([[4.21509616],
       [9.75532293]])
```

Computational Complexity

- Computational complexity
  - a mathematical notation that describes the limiting behavior of a function when the argument tends towards a particular value or infinity
  - Denoted using the big O notation
- The Normal Equation has computational complexity between $O(n^{2.4})$ and $O(n^3)$
- Under the hood the LinearRegression method of Scikit-Learn relies on SVD (Singular Value Decomposition) which is $O(n^2)$

Computational Complexity

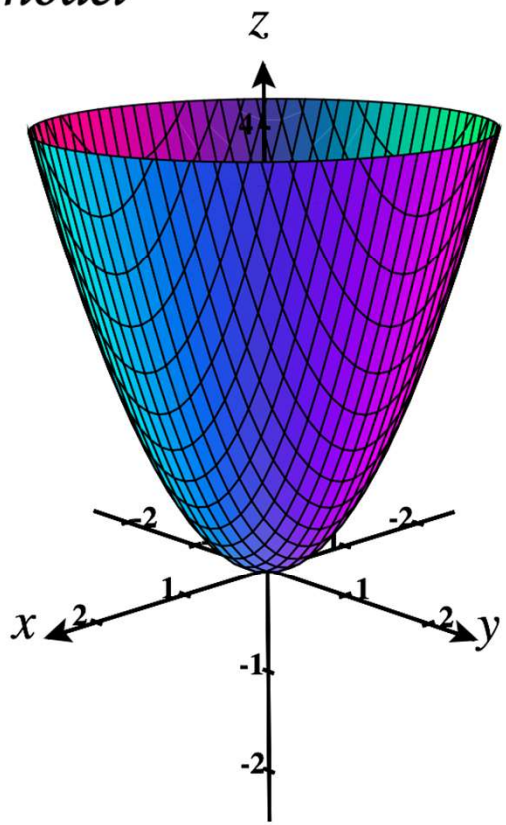
- The bad:
  - Training with the Normal Equation or SVD become very slow as the number of features grow

- The good:
  - Training complexity with respect to the training set size m is linear O(m)
  - Making predictions scales linearly with the number of instances and the number of features

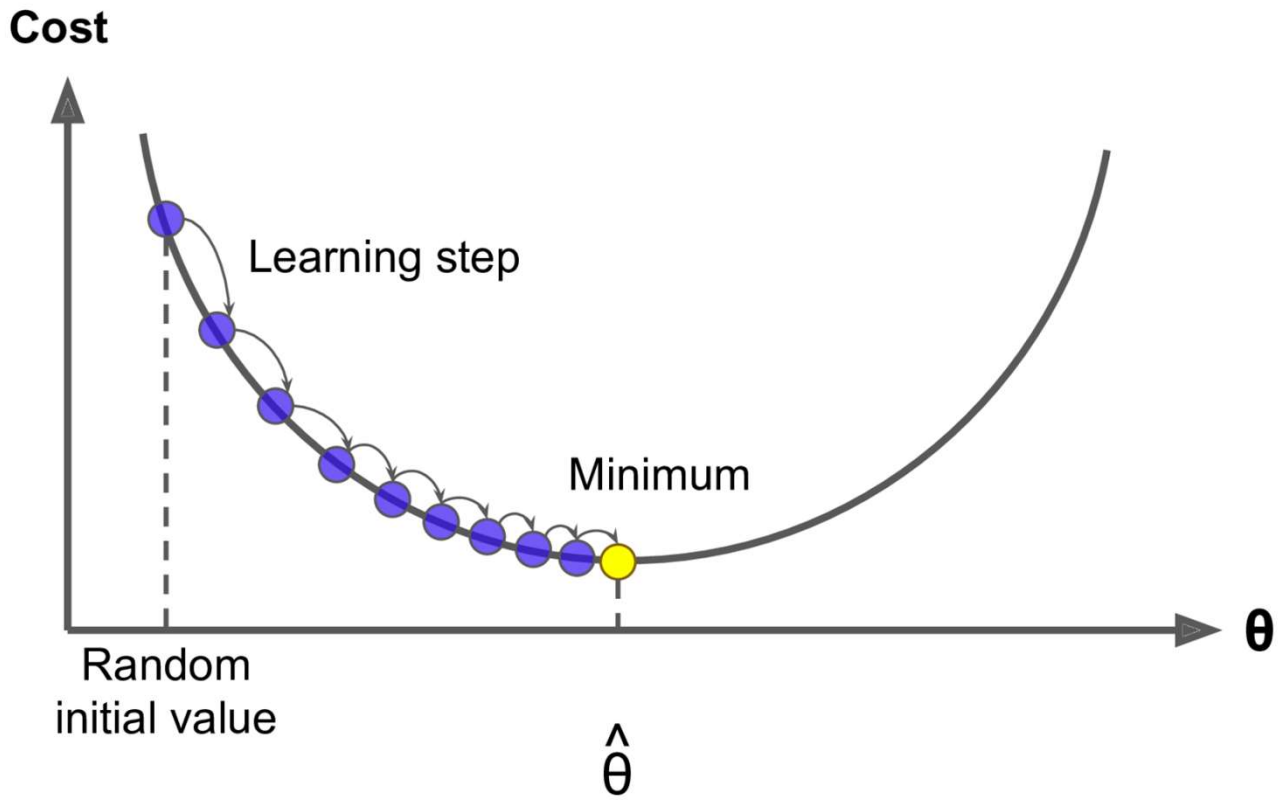
Gradient Descent (GD)

Equation 4-3. MSE cost function for a Linear Regression model

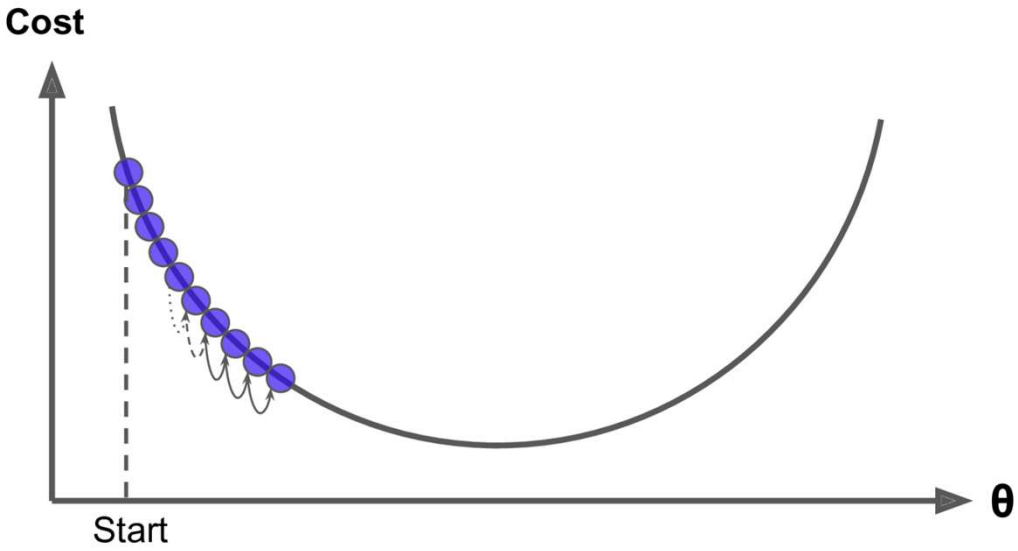
$$\mathrm{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2$$



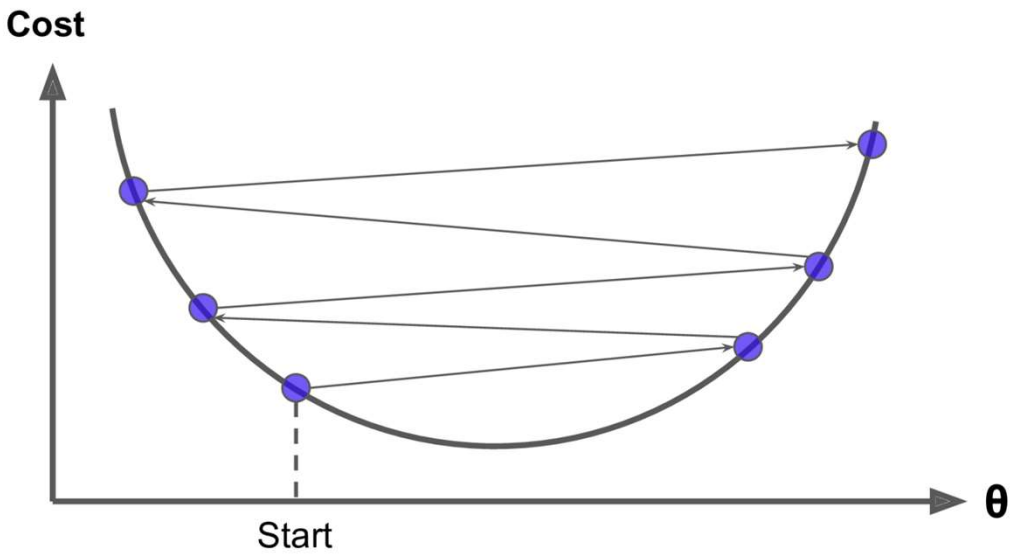
Gradient Descent (GD)



Gradient Descent (GD)

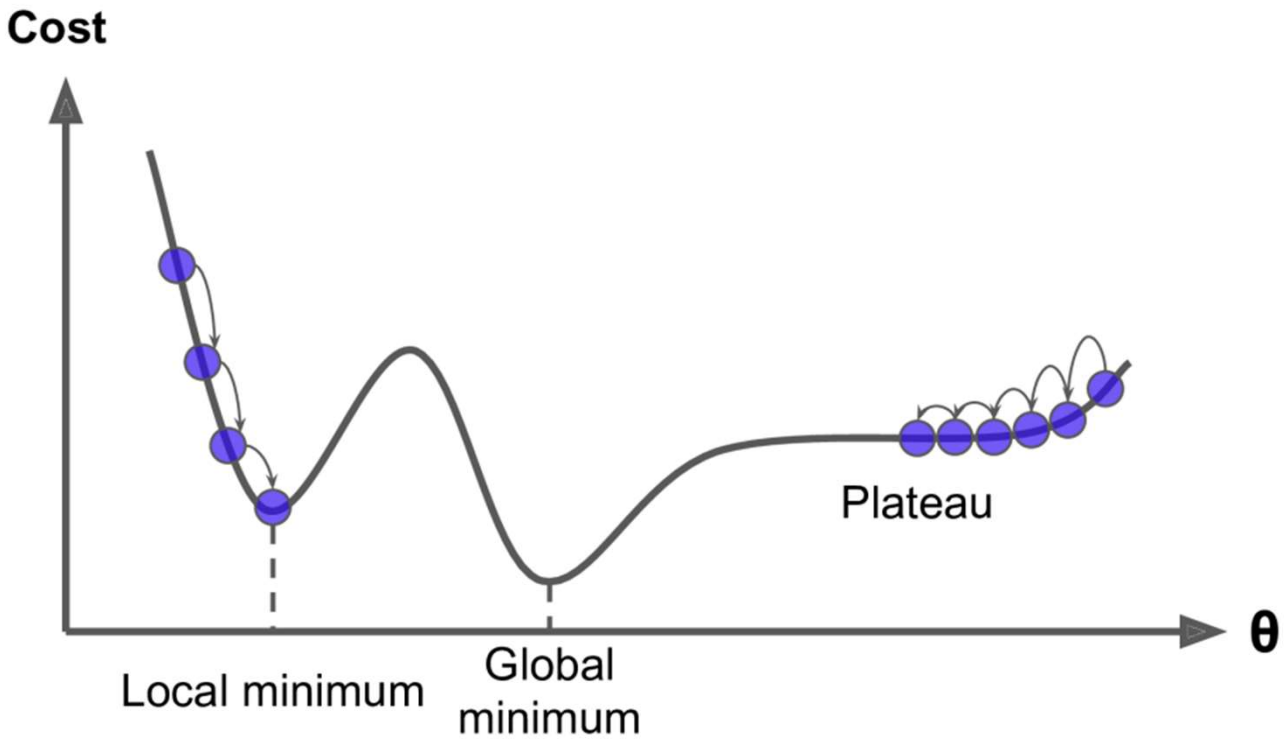


Learning rate too small

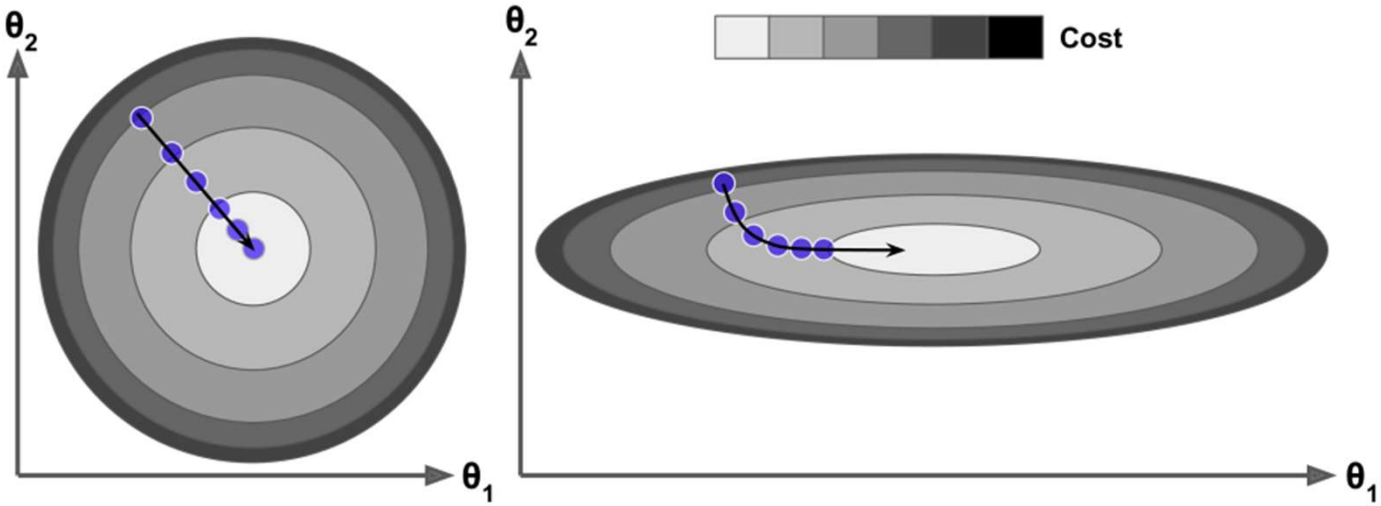


Learning rate too big

Gradient Descent (GD)



Gradient Descent (GD)



GD with and without feature scaling

Gradient Descent (GD)

- GD training
  - Search for minimum in the model parameter space over the training set
  - Higher number of parameters increases complexity of the search
  - Important to keep the number of parameters down

Batch Gradient Descent (BGD)

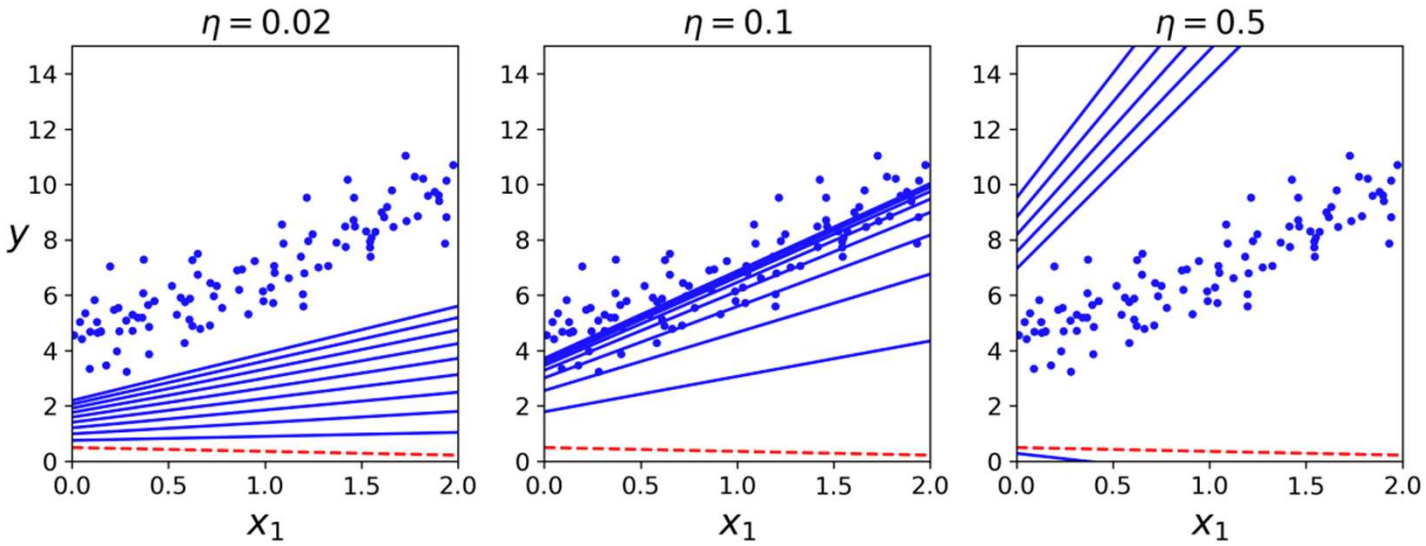
Equation 4-5. Partial derivatives of the cost function

$$\frac{\partial}{\partial \theta_j} \mathrm{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

- Every learning step involves calculating partial derivatives of each model weight theta
- Each partial derivative is calculated over entire training set (i.e. the entire training set batch)

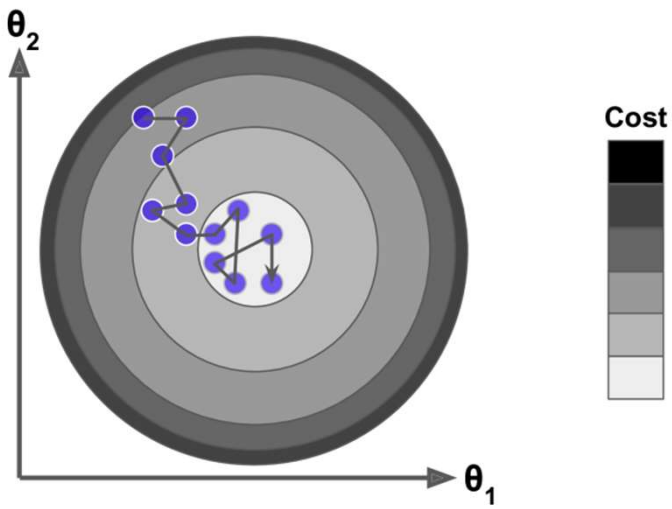
Batch Gradient Descent (BGD)

- $\nabla_{\boldsymbol{\theta}} \mathrm{MSE}(\boldsymbol{\theta})$ - gradient vector of the cost function
- $\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathrm{MSE}(\boldsymbol{\theta})$

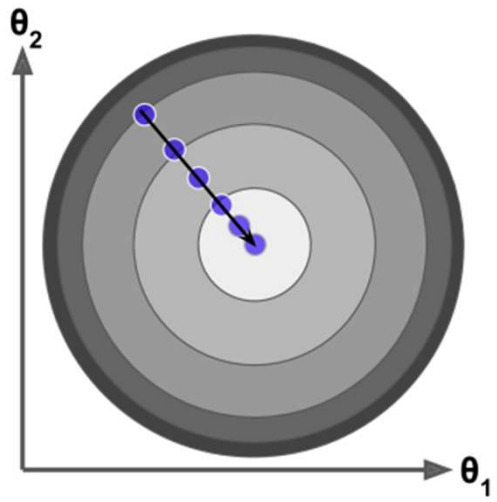


Stochastic Gradient Descent (SGD)

- Use a single random instance of the training set instead of the entire training set at each iteration.
- Less regular that BGD, but converges on average



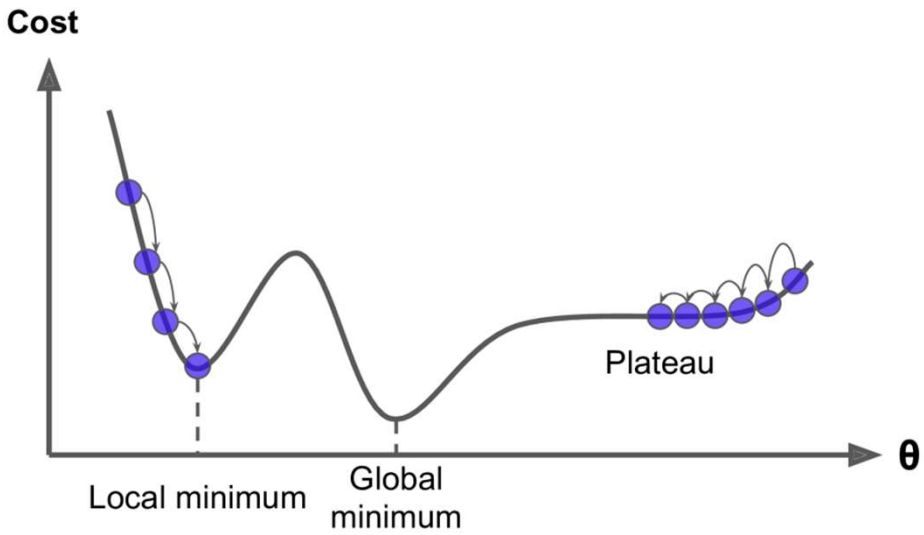
SGD



BGD

Stochastic Gradient Descent (SGD)

- Can "jump" out of local minima in cases of irregular cost functions



Stochastic Gradient Descent (SGD)

- To prevent SDG jumping around the global minimum
  - Use **learning schedule**
  - Result:

```
>>> theta
array([[4.21076011],
       [2.74856079]])
```

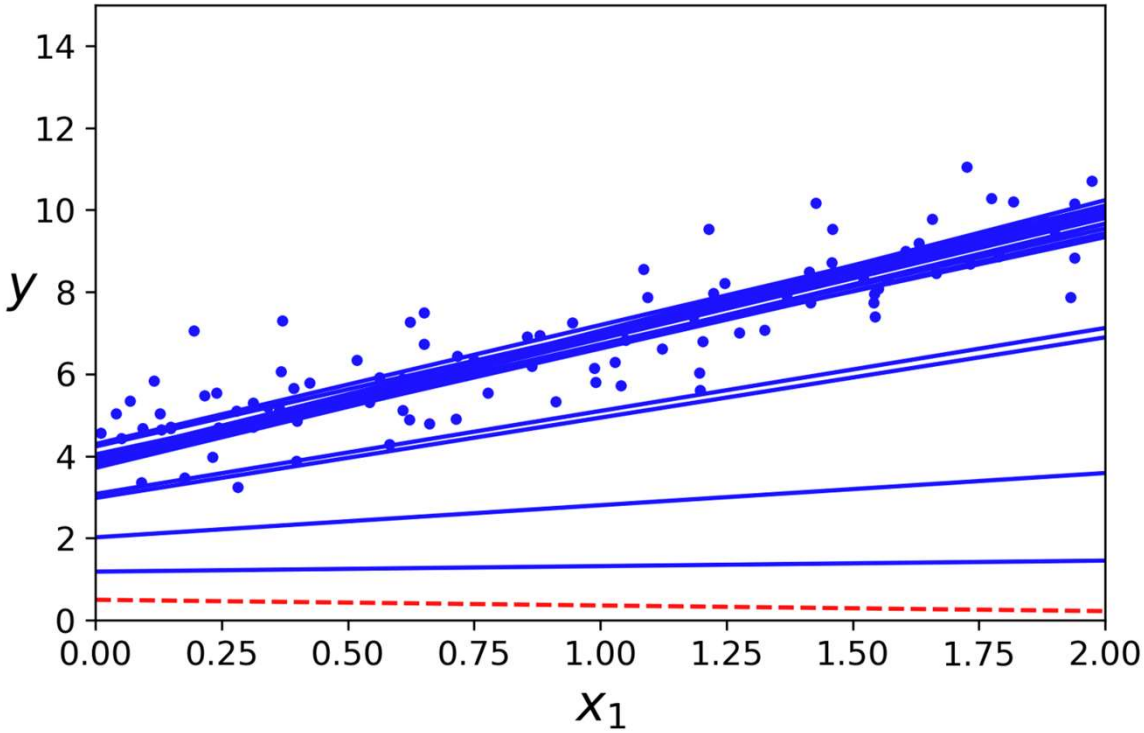
```python
n_epochs = 50
t0, t1 = 5, 50  # learning schedule hyperparameters

def learning_schedule(t):
    return t0 / (t + t1)

theta = np.random.randn(2,1)  # random initialization

for epoch in range(n_epochs):
    for i in range(m):
        random_index = np.random.randint(m)
        xi = X_b[random_index:random_index+1]
        yi = y[random_index:random_index+1]
        gradients = 2 * xi.T.dot(xi.dot(theta) - yi)
        eta = learning_schedule(epoch * m + i)
        theta = theta - eta * gradients
```

Stochastic Gradient Descent (SGD)



SDG the first 20 steps

Stochastic Gradient Descent (SGD)

- If instances are sorted by label, the algorithm will train for first label, then the second label, and so on

- Solution: Instances in the training set must be independently and identically distributed (IID)

Stochastic Gradient Descent (SGD)

- SGD using Scikit-Learn:

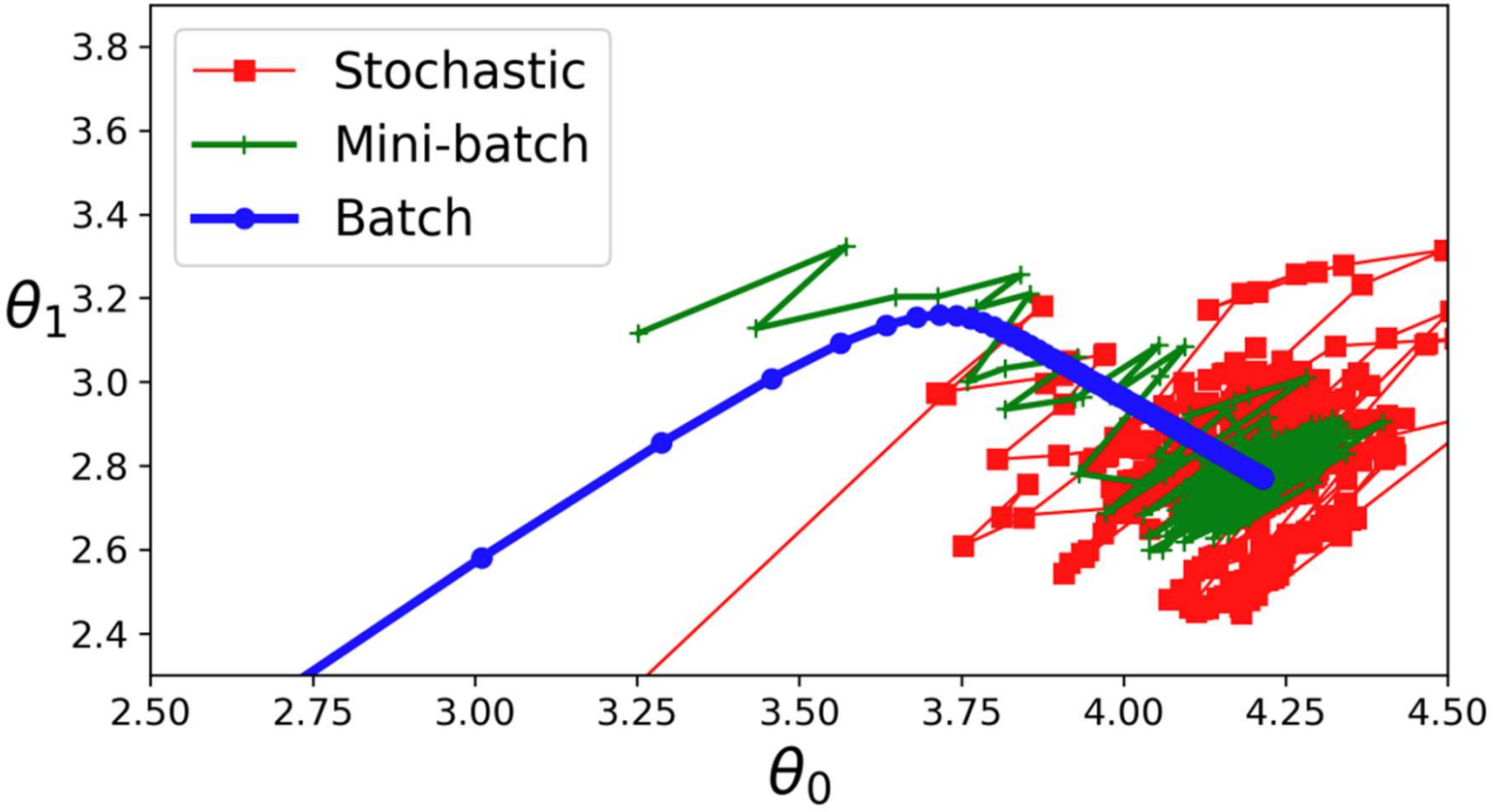
```
from sklearn.linear_model import SGDRegressor
sgd_reg = SGDRegressor(max_iter=1000, tol=1e-3, penalty=None, eta0=0.1)
sgd_reg.fit(X, y.ravel())
```

```
>>> sgd_reg.intercept_, sgd_reg.coef_
(array([4.24365286]), array([2.8250878]))
```

Mini-batch Gradient Descent

- Mini-batches of training instances picked at random at each step.
- Advantage: better performance on hardware-optimized matrix operations
- Disadvantage: can get stuck in local minima (compared with SGD)

Gradient Descent Comparison

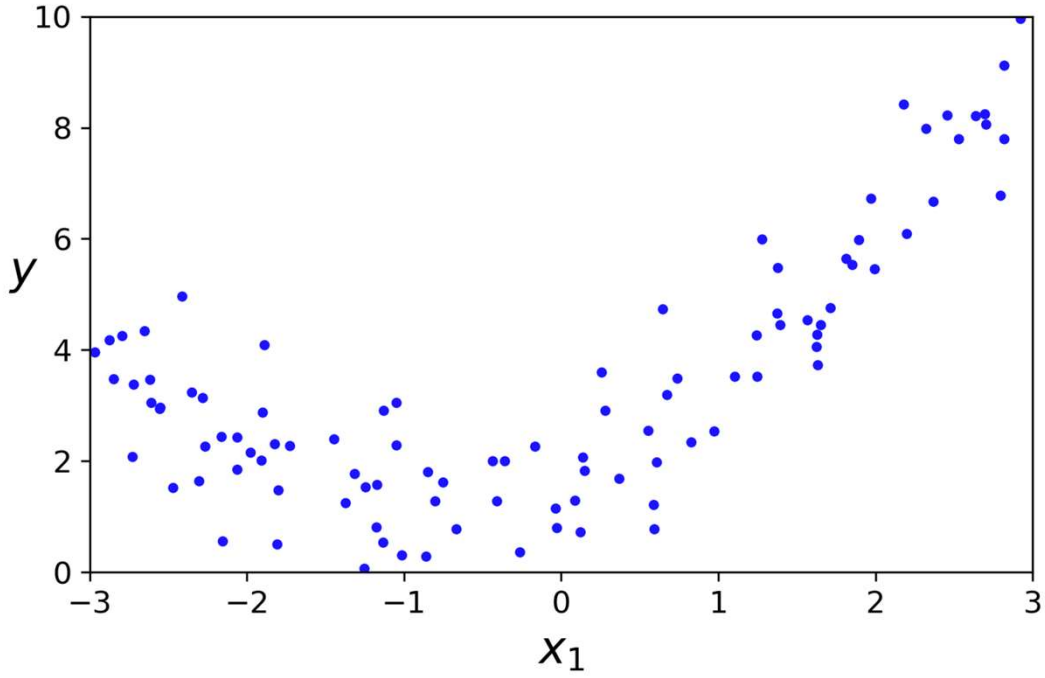


Linear Regression Algorithms Summary

Algorithm	Large *m*	Out-of-core support	Large *n*	Hyperparams	Scaling required	Scikit-Learn
Normal Equation	Fast	No	Slow	0	No	n/a
SVD	Fast	No	Slow	0	No	`LinearRegression`
Batch GD	Slow	No	Fast	2	Yes	`SGDRegressor`
Stochastic GD	Fast	Yes	Fast	≥2	Yes	`SGDRegressor`
Mini-batch GD	Fast	Yes	Fast	≥2	Yes	`SGDRegressor`

Polynomial Regression

```
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)
```



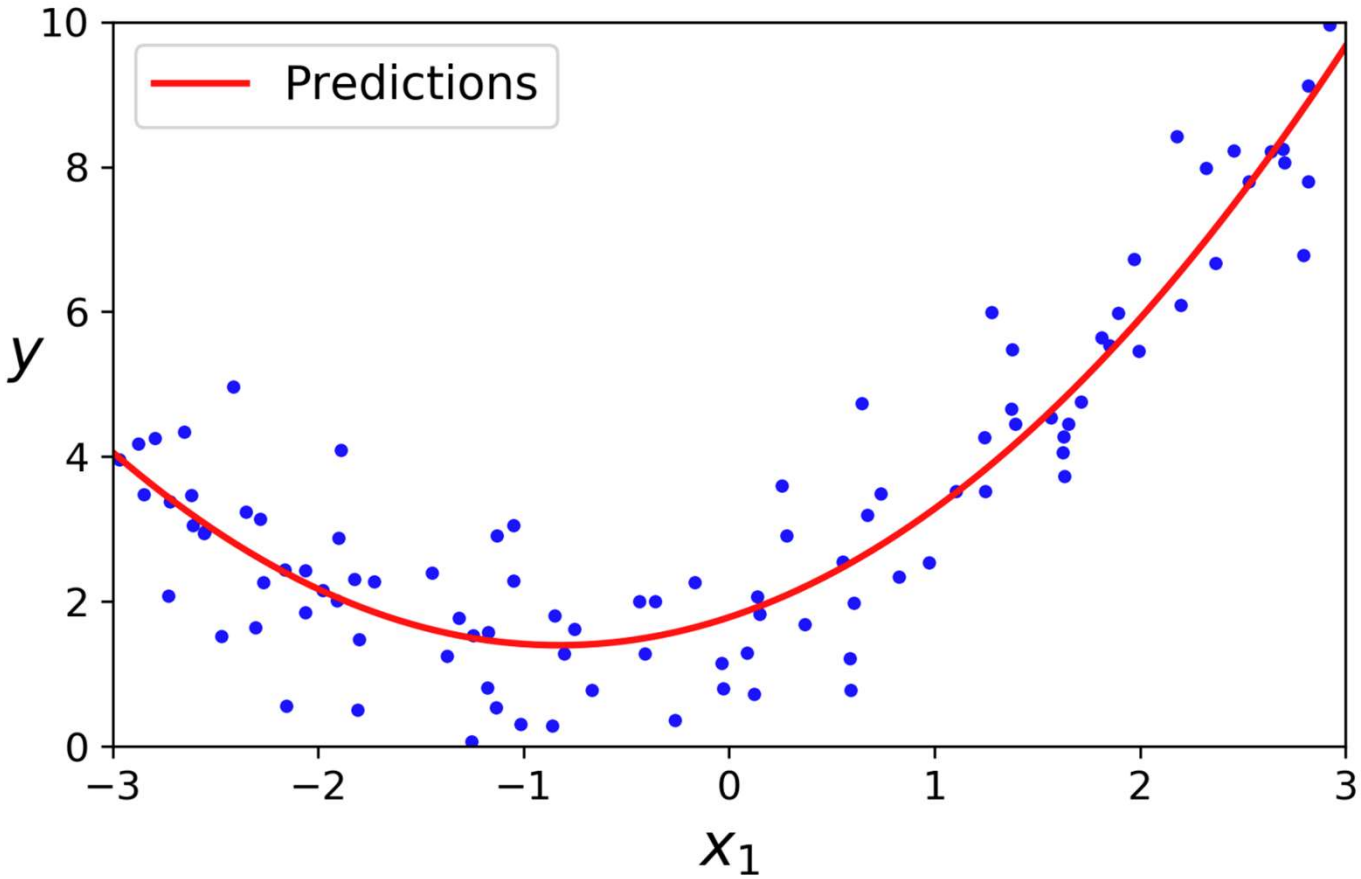
Polynomial Regression

$$y = ax^2 + bx + c$$

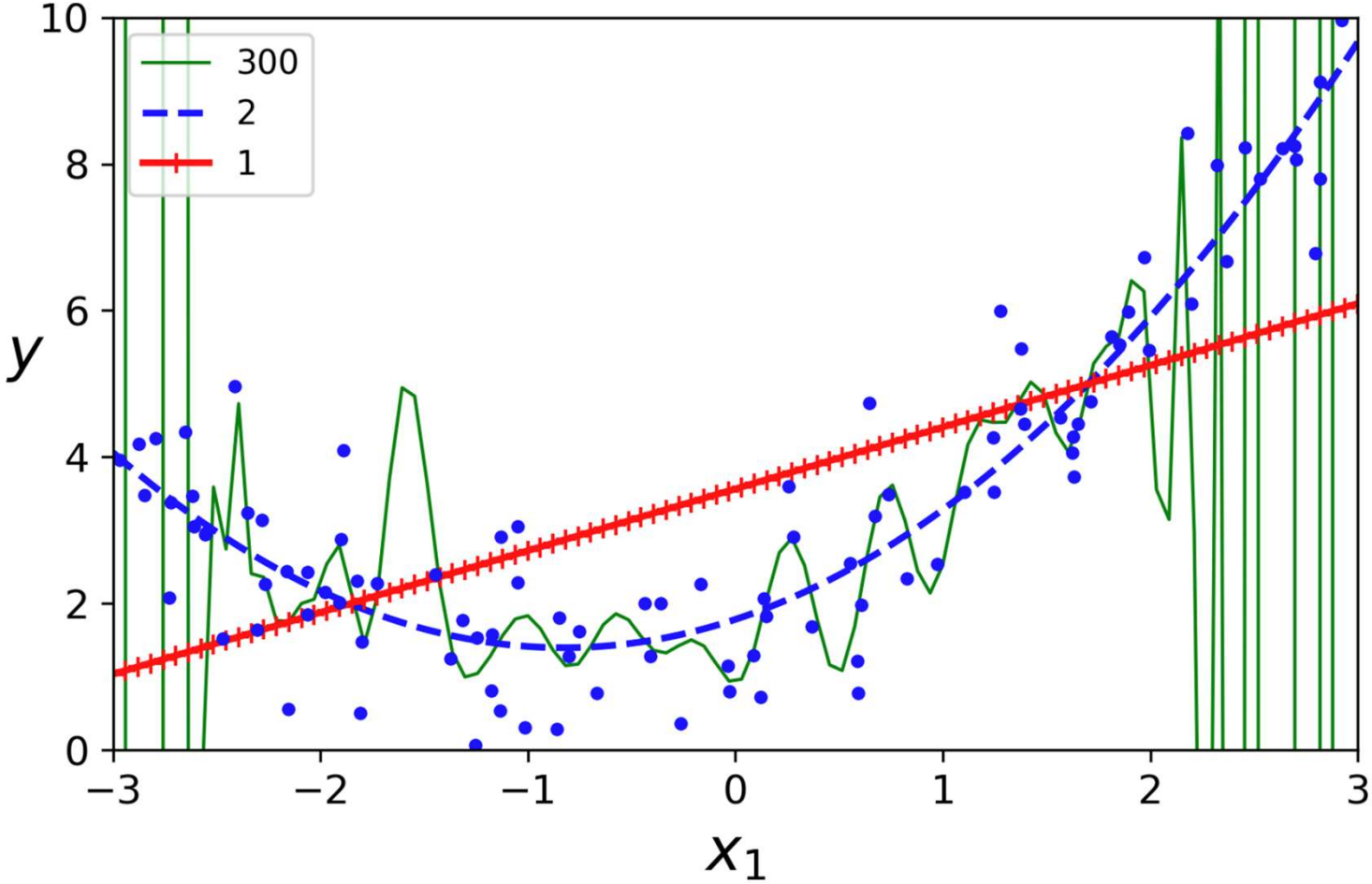
```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929, 0.56664654])
```

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

Polynomial Regression



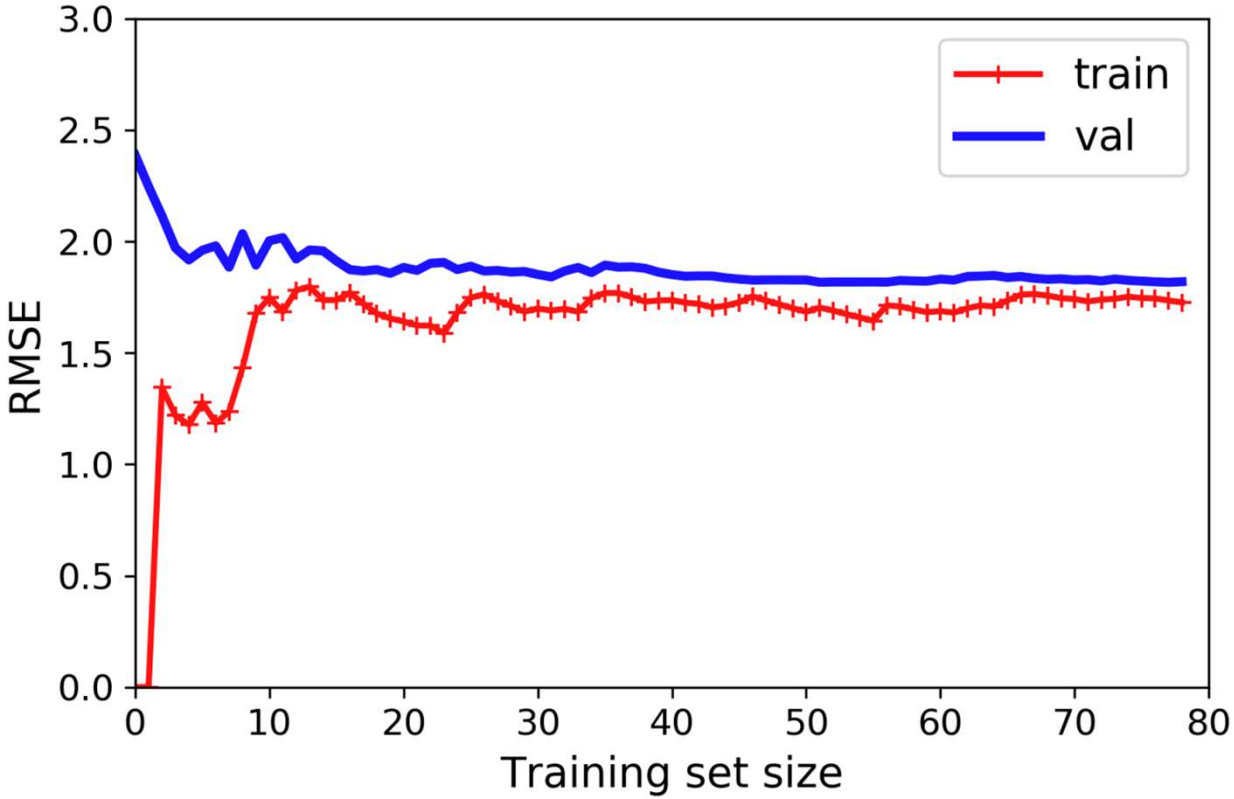
Polynomial Regression



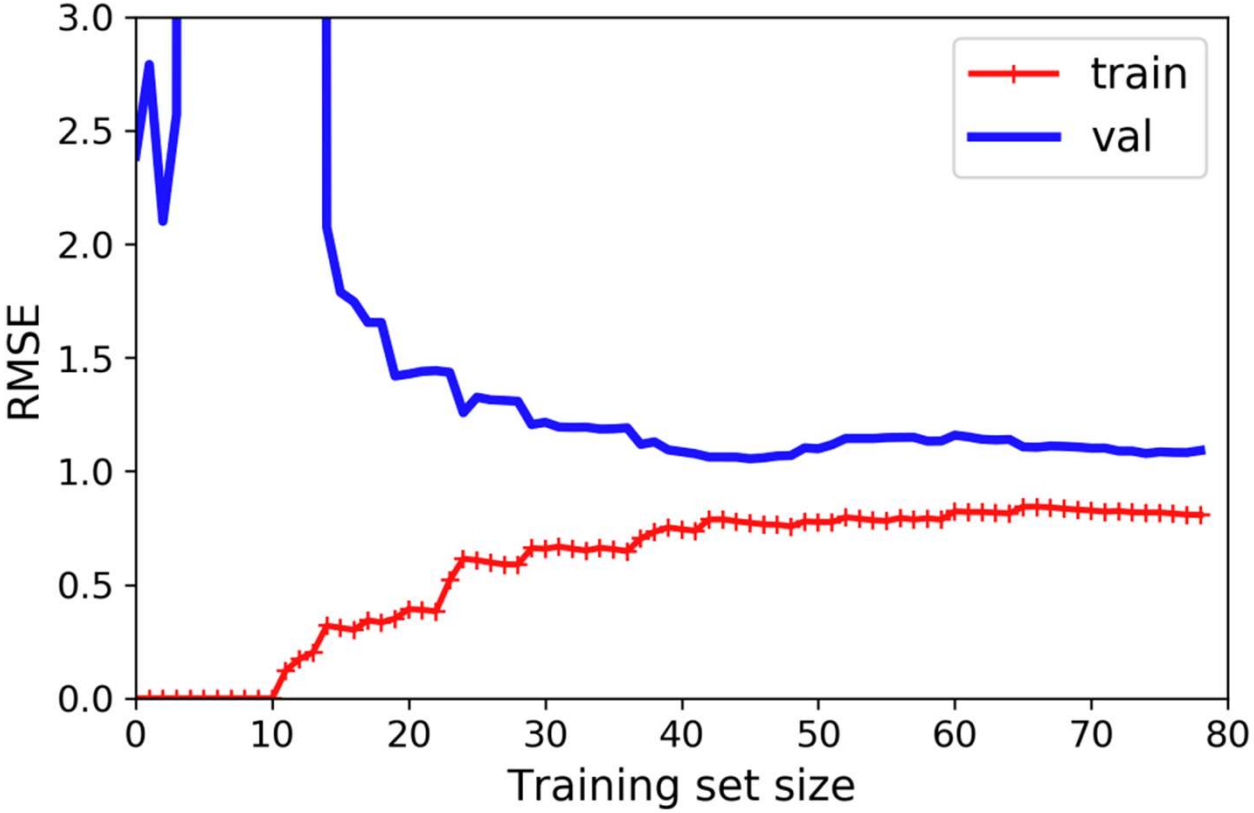
Learning Curves

- Measure the model performance as a function of the training set size
  - Break the training set into subsets of given size
  - Compare the cost function values for the training set and the validation set

Learning Curves – underfitting



Learning Curves – a better model



The Bias/Variance Tradeoff

Bias

This part of the generalization error is due to wrong assumptions, such as assuming that the data is linear when it is actually quadratic. A high-bias model is most likely to underfit the training data.[10]

Variance

This part is due to the model's excessive sensitivity to small variations in the training data. A model with many degrees of freedom (such as a high-degree polynomial model) is likely to have high variance, and thus to overfit the training data.

Irreducible error

This part is due to the noisiness of the data itself. The only way to reduce this part of the error is to clean up the data (e.g., fix the data sources, such as broken sensors, or detect and remove outliers).

Regularized Linear Models

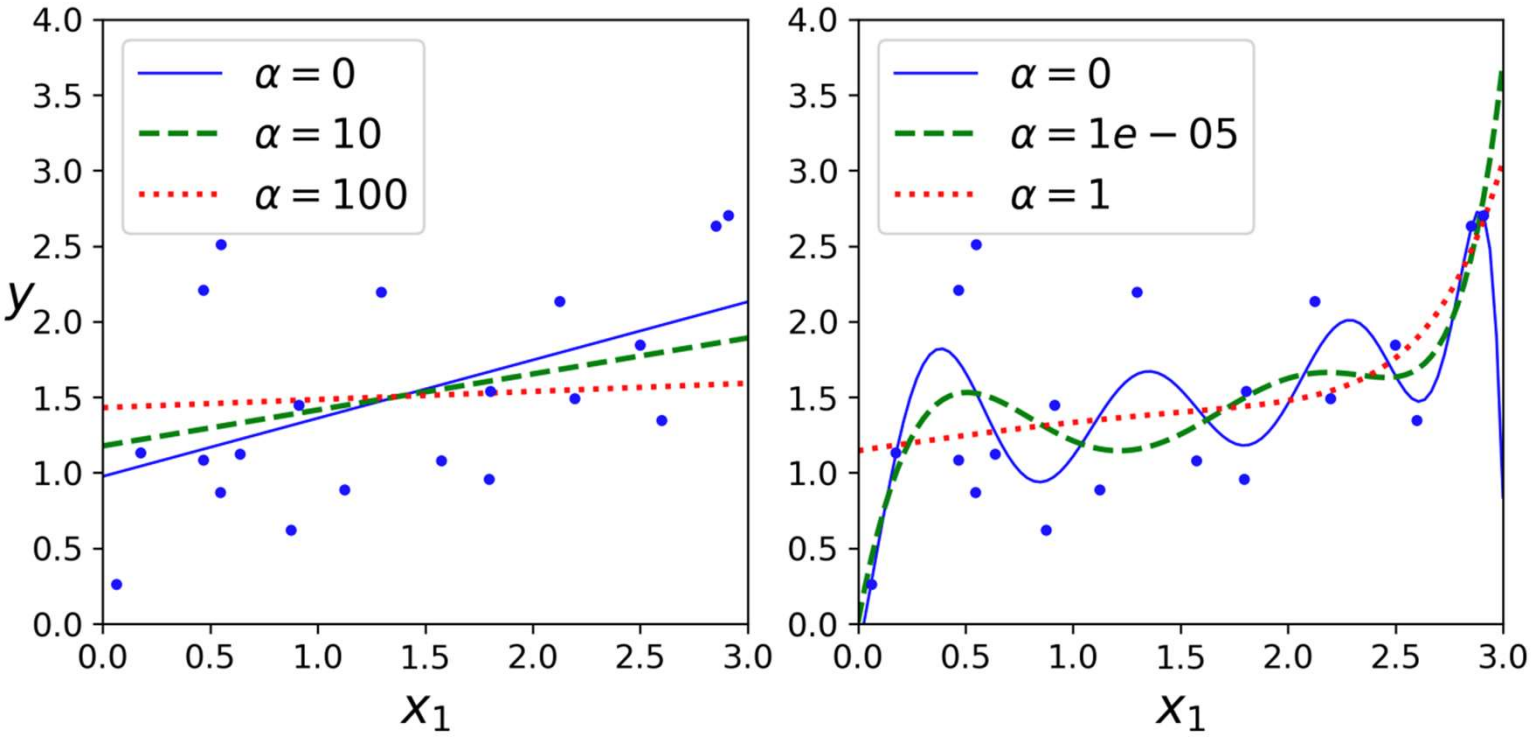
Ridge Regression

Lasso Regression

Elastic Net

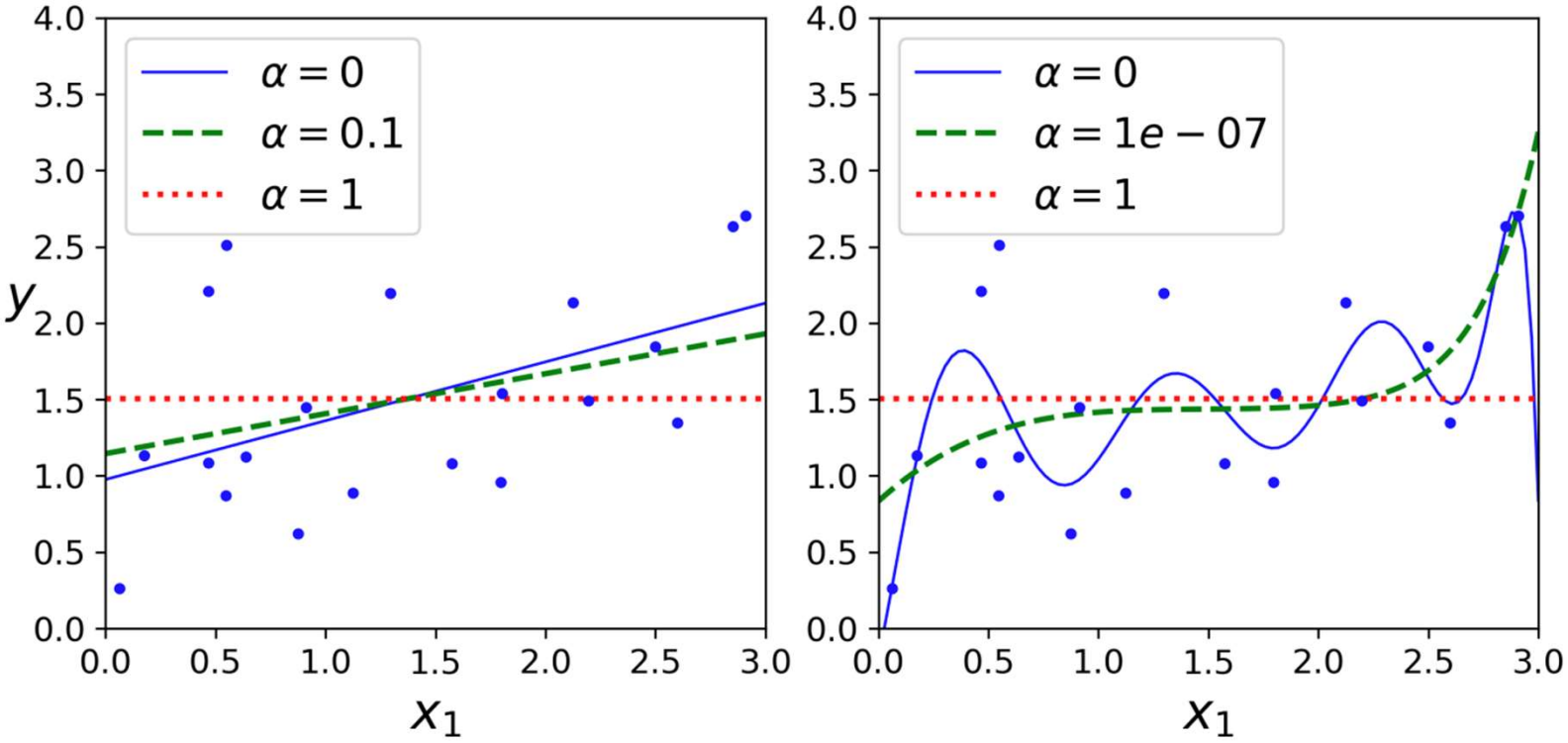
Ridge Regression

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \Sigma_{i=1}^{n} \theta_i^2$$



Lasso Regression

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \Sigma_{i=1}^{n} |\theta_i|$$



Elastic Net

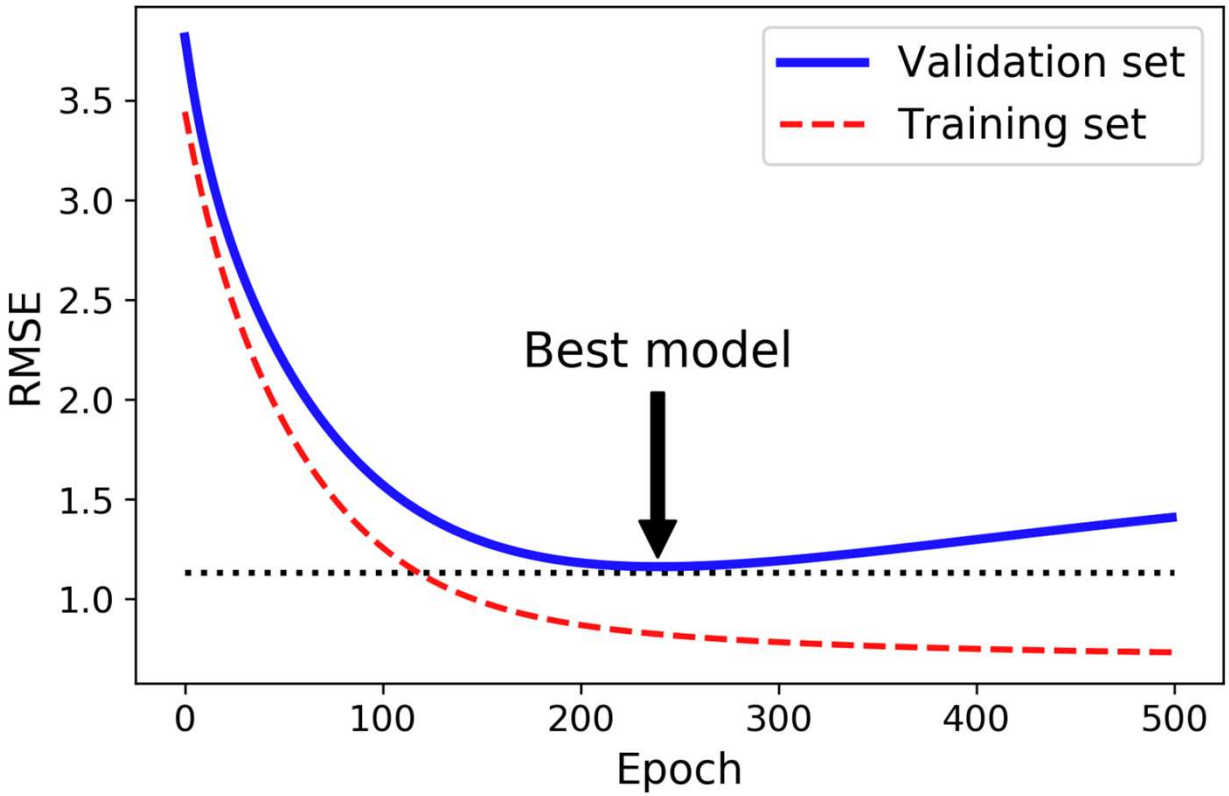
$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + r\alpha\Sigma_{i=1}^{n}\left|\theta_i\right| + \frac{1-r}{2}\alpha\Sigma_{i=1}^{n}\theta_i^{\,2}$$

Which Regularization To Use

- Pure linear regression is a bad idea
- If the data has only few features – use Lasso or Elastic Net
- If the data has many features – use Ridge
- Elastic Net is more stable than Lasso

Early Stopping

- Regularization method for iterative learning algorithms.
- Prevents the algorithms from overfitting
  - Manifestation: the training error goes down, but the validation error goes up



Early Stopping

```
from sklearn.base import clone

# prepare the data
poly_scaler = Pipeline([
        ("poly_features", PolynomialFeatures(degree=90, include_bias=False)),
        ("std_scaler", StandardScaler())
    ])
X_train_poly_scaled = poly_scaler.fit_transform(X_train)
X_val_poly_scaled = poly_scaler.transform(X_val)

sgd_reg = SGDRegressor(max_iter=1, tol=-np.infty, warm_start=True,
                       penalty=None, learning_rate="constant", eta0=0.0005)

minimum_val_error = float("inf")
best_epoch = None
best_model = None
for epoch in range(1000):
    sgd_reg.fit(X_train_poly_scaled, y_train)  # continues where it left off
    y_val_predict = sgd_reg.predict(X_val_poly_scaled)
    val_error = mean_squared_error(y_val, y_val_predict)
    if val_error < minimum_val_error:
        minimum_val_error = val_error
        best_epoch = epoch
        best_model = clone(sgd_reg)
```

Logistic Regression

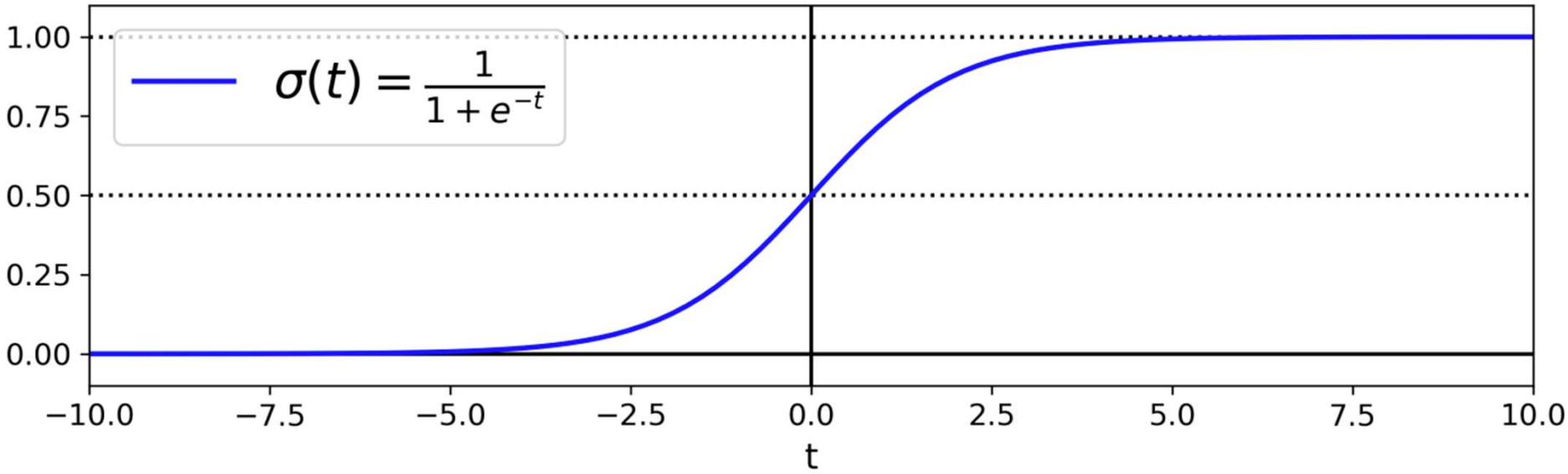
Estimating Probabilities

Equation 4-13. Logistic Regression model estimated probability (vectorized form)

$$\hat{p} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma\left(\mathbf{x}^T\boldsymbol{\theta}\right)$$

Equation 4-15. Logistic Regression model prediction

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

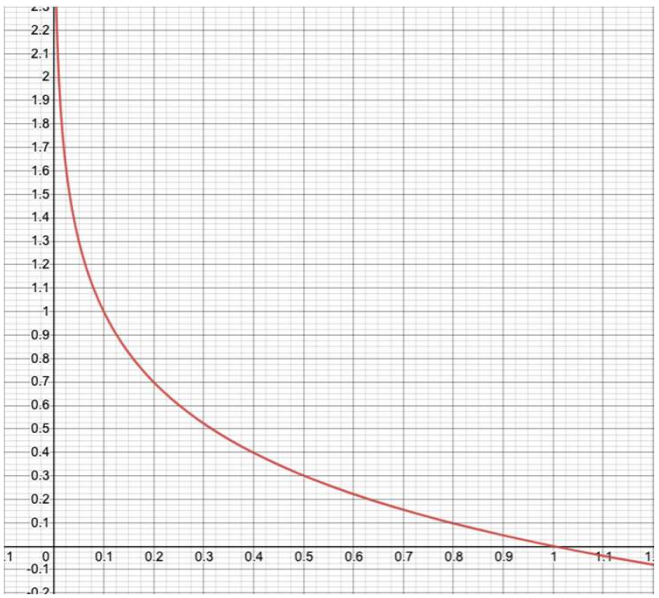


The Cost Function

Equation 4-16. Cost function of a single training instance

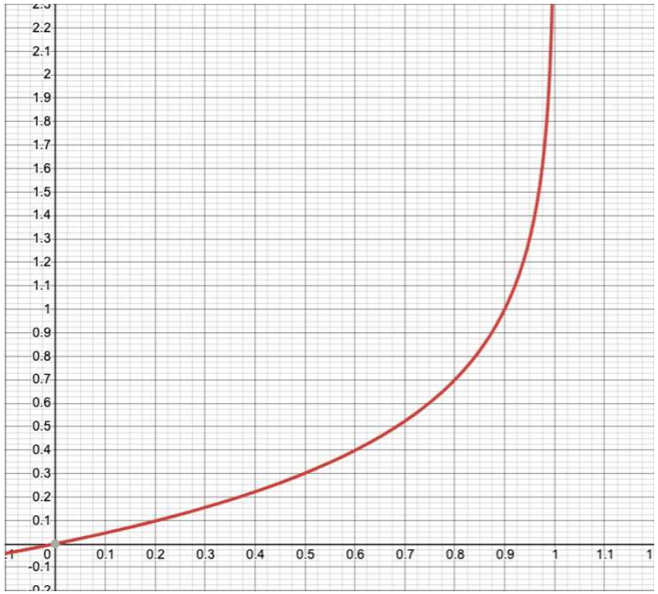
$$c(\boldsymbol{\theta}) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1 - \hat{p}) & \text{if } y = 0 \end{cases}$$

$-\log(\hat{p})$



$\hat{p}$

$-\log(1 - \hat{p})$



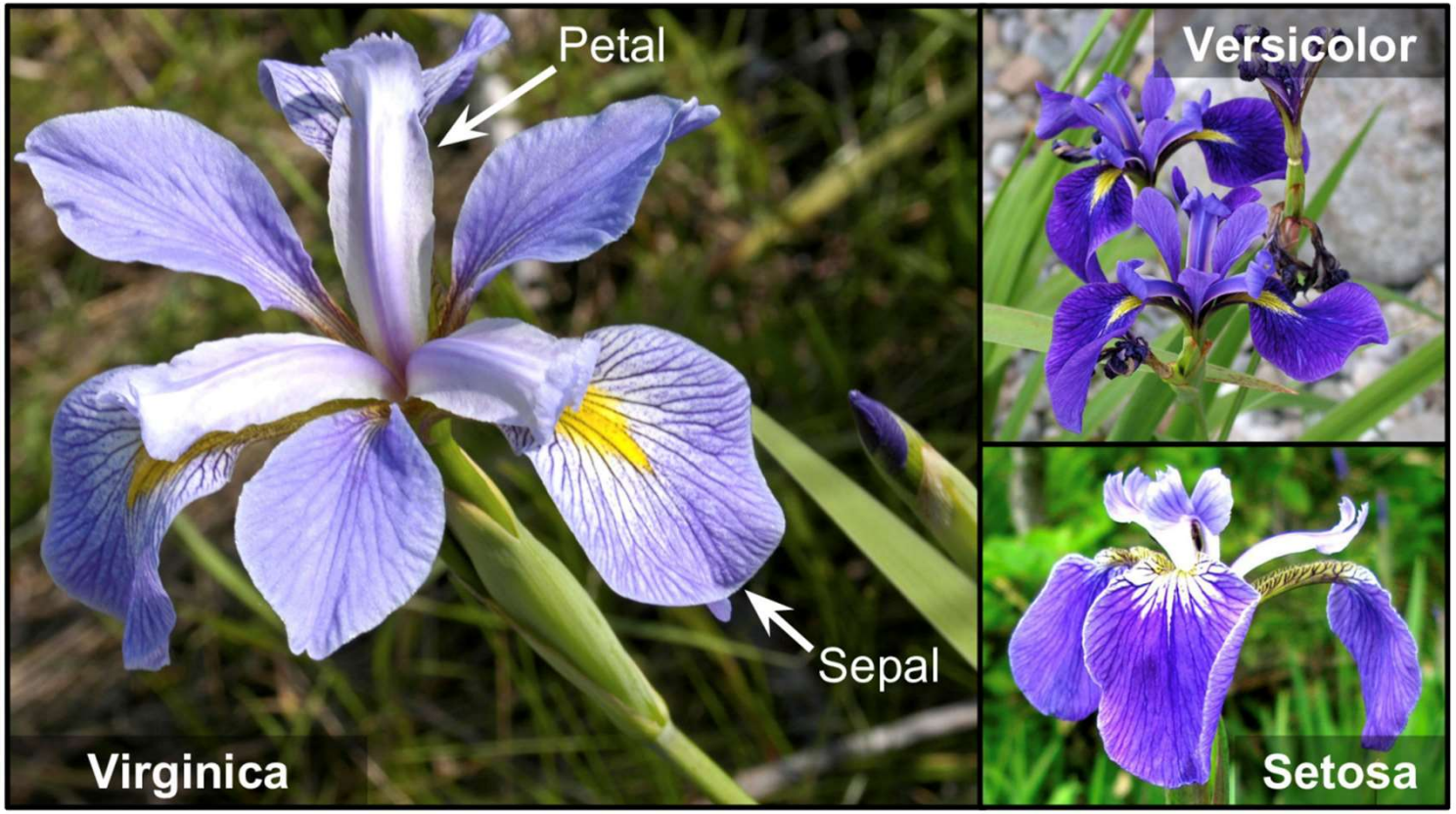
$\hat{p}$

The Cost Function

- No closed form solution

- Is convex
  - Use Gradient Descend (or any other optimization algorithm)

Decision Boundaries



Decision Boundaries

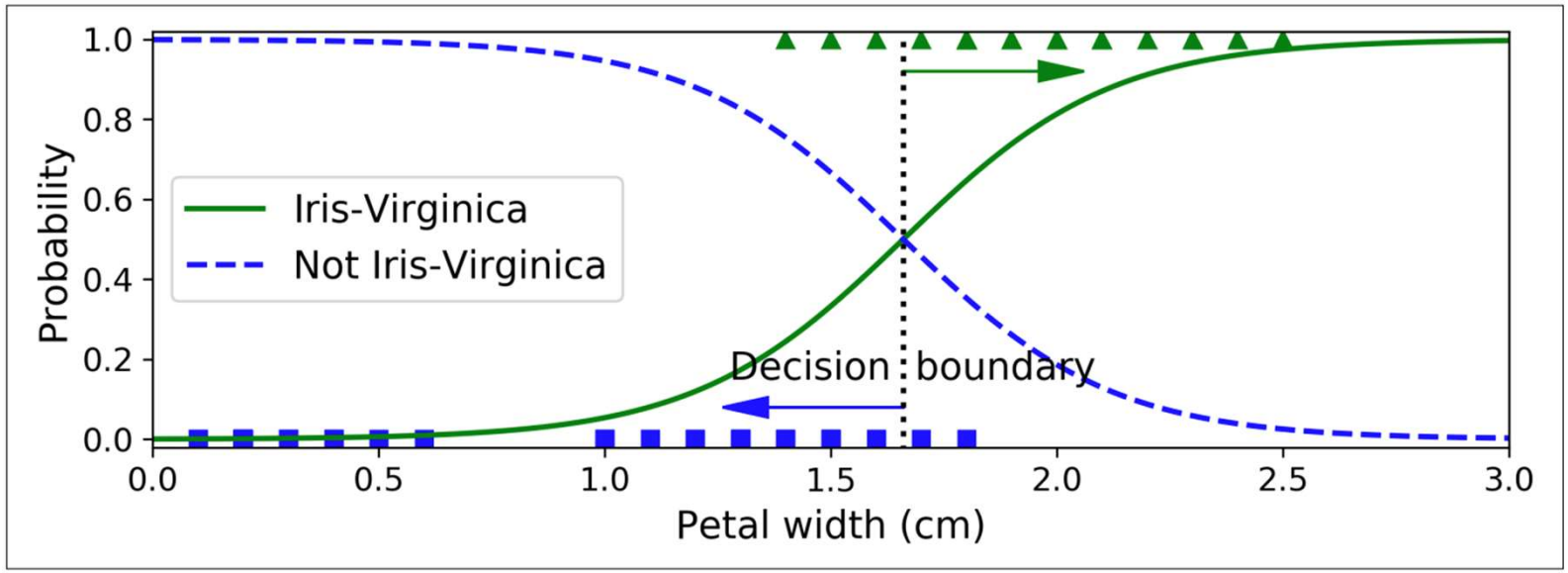
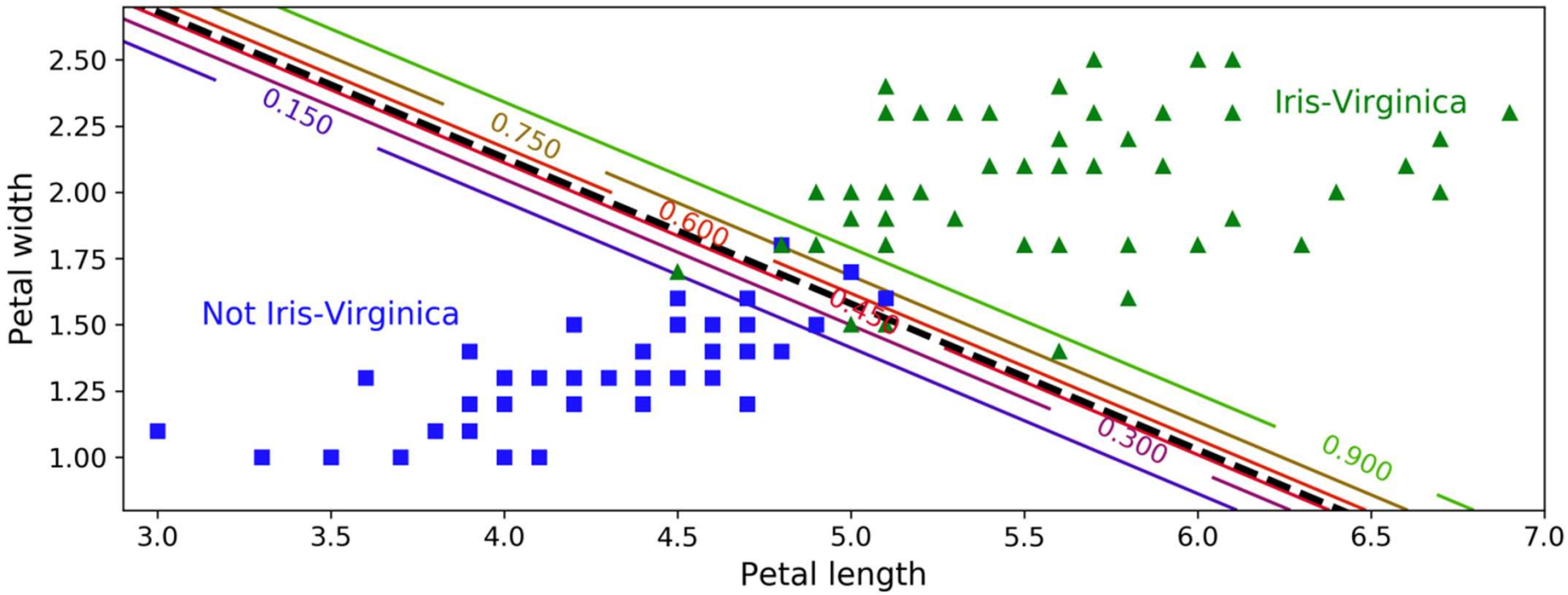
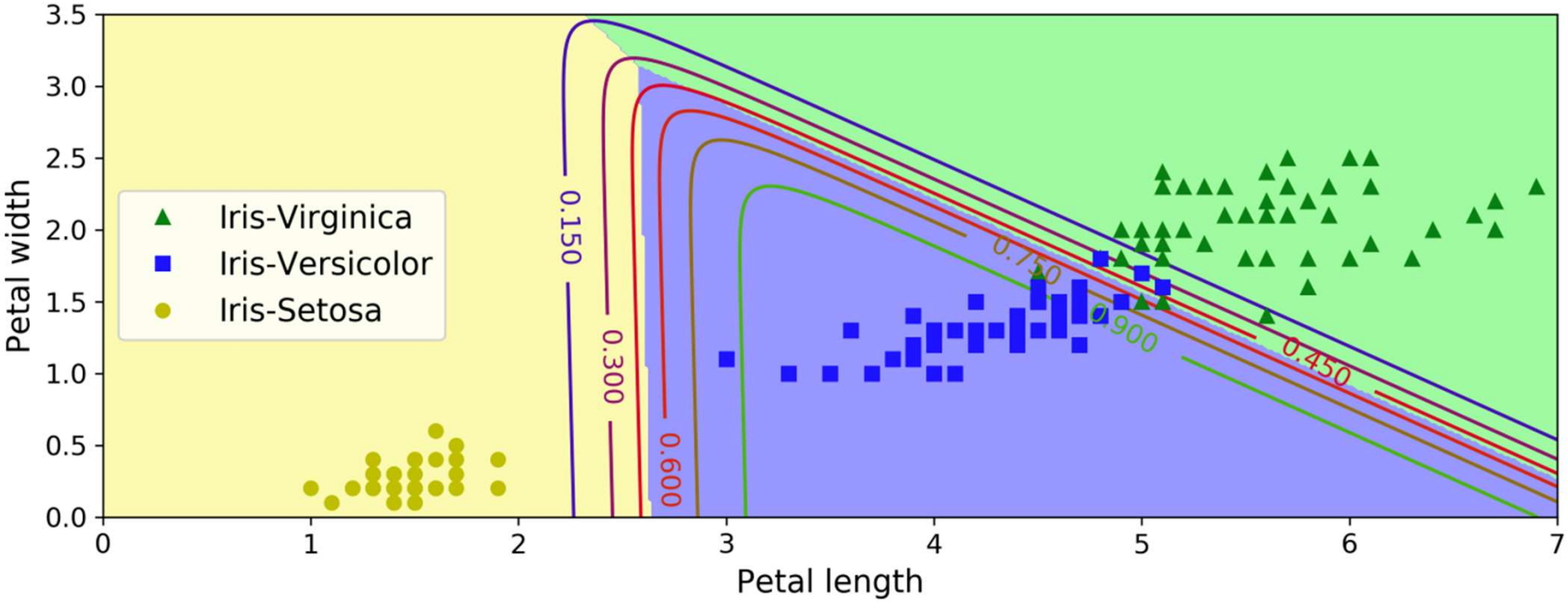


Figure 4-23. Estimated probabilities and decision boundary

```
>>> log_reg.predict([[1.7], [1.5]])
array([1, 0])
```

Decision Boundary



Softmax Regression